Reviewer Pack — Minimal Rank of Quaternion Algebras and AC0 Parity Circuit S…

Entry 75d870407726 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 01:28:59 UTC

Minimal Rank of Quaternion Algebras and AC0 Parity Circuit Size

Entry ID: `75d870407726` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 01:28:59 UTC

1. Conjecture

Field A (mathematical branch): Quaternion Algebra Theory **Field B** (complexity object): AC^0 Circuit Complexity

Statement:

> ['For any AC^0 parity circuit C with n inputs, the minimal rank of the quaternion algebra representing the Boolean function computed by C is upper bounded by a constant factor of log(size(C)).', 'There exists a constructive mapping that transforms any instance of an AC^0 parity circuit into a corresponding quaternion algebra such that the rank of this algebra provides a lower bound on the circuit size.', 'For all instances with n ≤ 40, if there exists no AC^0 parity circuit of size less than or equal to 2^(c*log(n)), then there is no quaternion algebra of minimal rank less than or equal to log(size(C)).']

Rationale (proposer's reasoning):

> ['Quaternion algebras provide a rich structure for studying the algebraic properties of boolean functions. The use of quaternion algebras in this context could potentially expose new insights into the complexity of parity computation.', 'The mapping between circuits and quaternion algebras is expected to reveal non-trivial connections between the algebraic properties of circuits and their computational complexity, especially for parity functions.', 'This conjecture aims to find a quantitative relation that bridges the gap between the abstract world of quaternion algebras and the concrete world of circuit complexity.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f44f5eeefca9b356`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The minimal rank of the quaternion algebra representing an AC^0 parity circuit is compared to log(size(C)) for circuits up to size 2^40.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	UNCERTAIN	1.00	HITS	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank quaternion algebra" AND "AC0 circuit complexity" - "quaternion algebra" AND "lower bound AC0 parity circuit" - "constructive mapping" AND "AC0 parity circuit to quaternion algebra"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def circuit_size(circuit):
        return len(circuit)

    def quaternion_algebra_rank(circuit):
        # Simplified mapping to a rank based on the number of inputs
        n = int(math.log2(len(circuit)))
        return n

    def log_size(size):
        return math.log(size, 2)

    results = []
    for _ in range(30):  # Ensure at least 30 instances per seed
        n = random.randint(5, 40)  # Sweep over different sizes
        circuit = generate_ac0_circuit(n)
        size = circuit_size(circuit)
        rank = quaternion_algebra_rank(circuit)
        log_s = log_size(size)

        results.append({
```

```
            "n": n,
            "circuit": circuit,
            "size": size,
            "rank": rank,
            "log_size": log_s
        })

    max_rank = max(result["rank"] for result in results)
    avg_log_size = sum(result["log_size"] for result in results) / len(results)

    conjecture_holds = max_rank <= avg_log_size * 2  # Upper bound factor of 2
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "max_rank",
        "metric_value": max_rank,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_max_rank = sum(result["metric_value"] for result in results) / len(results)
    std_max_rank = math.sqrt(sum((result["metric_value"] - mean_max_rank)**2 for result in results
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_max_rank} std={std_max_rank} support_fraction={suppor
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_max_rank} std={std_max_rank} support_fraction={suppor
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, making it impossible to determine if the conjecture is supported or falsified. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13034
2	propose	ollama_remote	glm4:latest	0	0	13862
3	preregistration	ollama_remote	glm4:latest	0	0	9034
4	novelty	ollama_remote	glm4:latest	0	0	8012
5	novelty	ollama_remote	glm4:latest	0	0	10650
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14253
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14724
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11551
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8113
10	judge	ollama_remote	glm4:latest	0	0	28797

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 132031 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/75d870407726.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/75d870407726.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/75d870407726.tar.gz` (if generated)